

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Manimaran

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I N. Murali Sai (192372309) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A
Short Answer Test

1. Suppose a software system is developed for a **Smart Banking System** where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

- Class
- Object
- Message Passing
- Encapsulation
- Polymorphism

(b) Describe the following object relationships:

- Association
- Aggregation
- Composition

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

(5 Marks)

Answer:

(a) Explain the following object-oriented concepts

1. Class

A **Class** is a blueprint or template that defines the properties (attributes) and behaviors (methods) of objects.

Example (Smart Banking System):

- **Class:** Customer
- **Attributes:** customerID, name, address, phoneNumber
- **Methods:** openAccount(), transferFunds(), payBill()

2. Object

An **Object** is an instance of a class that contains actual data and can perform the actions defined in the class.

Example:

- **Object:** Customer1
- **Name:** Sayed Fazal

- **Account Number:** 10024567
- **Balance:** \$5,000

This object can transfer funds and pay bills.

3. Message Passing

Message Passing is the communication between objects through method calls. One object requests another object to perform a task.

Example:

When a customer transfers money:

- Customer object sends a request to the Account object.
- The Account object checks the balance and transfers the amount.
- The Transaction object records the transaction.

4. Encapsulation

Encapsulation is the process of hiding data and allowing access only through public methods, ensuring data security.

Example:

The account balance is declared as a private variable.

Private:

balance

Public Methods:

deposit()

withdraw()

getBalance()

5. Polymorphism

Polymorphism allows the same method to perform different tasks depending on the object calling it.

Example:

Method:

calculateInterest()

Different account types calculate interest differently:

- Savings Account → 5% interest

- Current Account → 0% interest
- Fixed Deposit → 7% interest

The same method behaves differently for each account type.

(b) Describe the following object relationships

1. Association

Association is a relationship where two independent objects communicate with each other.

Example:

- Customer ↔ Bank Account

A customer owns one or more bank accounts, but both can exist independently.

Customer ----- BankAccount

2. Aggregation

Aggregation is a "has-a" relationship where one object contains another object, but the contained object can exist independently.

Example:

- Bank → Employees

Employees continue to exist even if a particular bank branch is closed.

Bank ◇----- Employee

(◇ = Hollow Diamond)

3. Composition

Composition is a strong "part-of" relationship where the contained object cannot exist without its parent object.

Example:

- Bank Account → Transaction History

If the bank account is deleted, its transaction history is also deleted.

BankAccount ◆----- Transaction

(◆ = Filled Diamond)

(c) How these concepts improve software reusability and maintainability

OOP Concept	Benefit in Smart Banking System
Class	Reusable blueprint for creating multiple customers, accounts, and transactions.
Object	Represents real-world banking entities, making the system easier to understand.
Message Passing	Enables communication between different modules without tight coupling.
Encapsulation	Protects sensitive banking information such as account balance and PIN from unauthorized access.
Polymorphism	Allows the same operation (e.g., calculateInterest()) to work differently for different account types, reducing duplicate code.
Association	Models relationships like Customer–Account without making objects dependent on each other's lifecycle.
Aggregation	Allows independent objects such as employees or customers to be reused in different parts of the system.
Composition	Maintains data integrity by ensuring dependent objects (e.g., transaction history) exist only with their parent account.

2. Consider developing a **Smart Healthcare Management System**.

(a) Explain the following object-oriented design principles:

- Abstraction
- Encapsulation
- Modularity

(b) Describe the following SOLID principles:

- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)

(c) Explain how these principles improve software quality and flexibility.

(5 Marks)

Answer:

(a) Object-Oriented Design Principles

1. Abstraction

- Hides implementation details and shows only essential features.
- **Example:** Doctors access patient records without knowing database operations.

2. Encapsulation

- Protects data by restricting direct access through methods.
- **Example:** Patient records are accessed using get() and set() methods.

3. Modularity

- Divides the system into independent modules.
- **Example:** Patient, Appointment, Billing, and Pharmacy modules.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

- A subclass should be replaceable for its parent class without affecting program behavior.
- **Example:** Cardiologist can replace Doctor.

2. Interface Segregation Principle (ISP)

- Classes should implement only the interfaces they need.
- **Example:** Billing and Appointment interfaces are separate.

(c) How these principles improve software quality and flexibility

Principle	Improvement in Smart Healthcare Management System
Abstraction	Hides complex implementation details, making the system easier for doctors, nurses, and administrators to use.
Encapsulation	Protects sensitive patient records by restricting direct access and allowing controlled updates.
Modularity	Separates the system into independent modules, making development, testing, and maintenance easier.
Liskov Substitution Principle (LSP)	Allows specialized healthcare professionals (e.g., Cardiologist, Dermatologist) to replace a general doctor object without changing program behavior.
Interface Segregation Principle (ISP)	Ensures each user or component implements only the required functions, reducing unnecessary dependencies and improving maintainability.

3. Suppose a software company is developing an **Online Food Delivery System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Planning
- Requirement Analysis
- Design
- Development
- Testing

- Maintenance

(b) Describe the purpose of each SDLC phase.

(c) Explain how SDLC contributes to delivering reliable software.

(5 Marks)

Answer:

(a) SDLC Phases

1. Planning

- Defines project goals, scope, budget, and timeline.

2. Requirement Analysis

- Collects and analyzes user and business requirements.

3. Design

- Creates the system architecture, database, and user interface design.

4. Development

- Developers write the code based on the design.

5. Testing

- Identifies and fixes bugs to ensure the software works correctly.

6. Maintenance

- Updates the software, fixes issues, and adds new features after deployment.

(b) Purpose of Each SDLC Phase

- **Planning:** Determines project feasibility and objectives.
- **Requirement Analysis:** Ensures all user needs are clearly understood.
- **Design:** Provides a blueprint for system development.
- **Development:** Converts the design into a working application.
- **Testing:** Ensures software quality and reliability.
- **Maintenance:** Keeps the software updated and functioning efficiently.

(c) How SDLC contributes to delivering reliable software

SDLC Phase	Contribution to Reliable Software
Planning	Defines clear goals and reduces project risks.

SDLC Phase	Contribution to Reliable Software
Requirement Analysis	Ensures the software meets user needs.
Design	Creates a well-structured and scalable system.
Development	Produces quality code following the design.
Testing	Detects and fixes errors before release.
Maintenance	Improves performance, fixes bugs, and keeps the software updated.

4. Consider a project for developing a **Smart Agriculture Monitoring System**, where customer requirements change regularly.

(a) Explain the following SDLC models:

- Prototype Model
- Spiral Model
- Agile Model
- DevOps Model

(b) Compare the Prototype Model and Agile Model.

(c) Recommend the most suitable SDLC model for this project with justification.

(5 Marks)

Answer:

(a) SDLC Models

1. Prototype Model

- A working prototype is developed first to gather user feedback before building the final system.

2. Spiral Model

- Development is carried out in repeated cycles with risk analysis at each stage.

3. Agile Model

- Software is developed in small iterations with continuous customer feedback and frequent updates.

4. DevOps Model

- Combines development and operations to enable continuous integration, testing, deployment, and maintenance.

(b) Comparison: Prototype Model vs Agile Model

Prototype Model	Agile Model
Develops a prototype first.	Develops the software in small iterations.
Used to clarify requirements.	Used for continuous development and improvement.
Customer feedback is mainly on the prototype.	Customer feedback is taken after every iteration.
Best for unclear requirements.	Best for changing requirements.

(c) Most Suitable SDLC Model

Recommended Model: Agile Model

Justification:

- Handles frequent requirement changes effectively.
 - Supports continuous customer feedback.
 - Delivers features quickly through iterations.
 - Easy to update and improve the system throughout development.
-

5. Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Explain the importance of UML in object-oriented software development.

(b) Describe the following UML diagrams:

- State Diagram
- Component Diagram
- Deployment Diagram
- Package Diagram

(c) Explain how UML models improve communication among stakeholders.

(5 Marks)

Answer:

(a) Importance of UML in Object-Oriented Software Development

- Provides a visual representation of the system.
- Helps in planning and designing software.
- Improves communication among developers and stakeholders.

- Identifies design issues before coding.
- Makes maintenance and future enhancements easier.

(b) UML Diagrams

1. State Diagram

- Shows the different states of an object and how it changes over time.
- **Example:** Bus → Available → Booked → In Transit → Completed.

2. Component Diagram

- Shows the software components and their relationships.
- **Example:** User Module, Booking Module, Payment Module, Database.

3. Deployment Diagram

- Shows the physical deployment of software on hardware devices.
- **Example:** Client Device → Web Server → Application Server → Database Server.

4. Package Diagram

- Organizes related classes into packages to improve structure.
- **Example:** User Package, Booking Package, Payment Package.

(c) How UML models improve communication among stakeholders

- Provides a common visual language for everyone.
- Helps developers understand the system design.
- Makes customer requirements easier to explain.
- Reduces misunderstandings and design errors.
- Improves collaboration among developers, testers, and clients.

6. Consider developing a Smart Inventory Management System.

(a) Explain the concept of modular software design.

(b) Compare the procedural programming approach with the object-oriented programming approach.

(c) Explain how modularity improves software scalability, maintainability, and reusability.

(5 Marks)

Answer:

(a) Concept of Modular Software Design

- Modular software design divides a system into smaller, independent modules.
- Each module performs a specific function and can be developed and tested separately.
- **Example:** Inventory, Supplier, Sales, and Billing modules.

(b) Procedural Programming vs Object-Oriented Programming

Procedural Programming	Object-Oriented Programming (OOP)
Focuses on functions/procedures.	Focuses on objects and classes.
Data and functions are separate.	Data and methods are combined.
Less reusable.	Highly reusable through inheritance and polymorphism.
Harder to maintain for large systems.	Easier to maintain and extend.

(c) How modularity improves software quality

- **Scalability:** New modules can be added without affecting existing ones.
- **Maintainability:** Individual modules can be updated or fixed easily.
- **Reusability:** Modules can be reused in other projects, reducing development time.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps